DEVOPS ASSIGNMENT-01

CHINTA SRI DURGA ROHIT
23FE10CSE00596

1. Evaluate Devops pipelines for data-Intensive applications.

Ans. Data Intensive applications (ETL, ML, analytics streaming) differ from typical app pipelines.

Key evalution criteria:

- Data versioing : Use DVC (Data version control) for sharing ML models, Delta Lake Lake FS for dataset version control.
- Reproducibility : pipeline must recreate exact data state and model artifacts
- Scalability : Distributed compute (Spark, Flink, Kubernetes Jobs)
- Artifact Management : store models, schemas, and processed data seperately
- Data Quality Gates : Automated validation using Great Expections or Custom checks
- Performance Testing : Load Testing for datasets, E2E test for web application, Integration testing,
- Rollback Testing : versioned data + container images
- Observebility : Logs, metrics, lineage tracking (e.x. OpenLineage)

A good pipeline treats data as a first class artifact, not just as as a code.

2. Git versioning strategies for a schema evolution.

Ans. Schema Changes must be visible which means it must be traceable and backward compatible.

Common Strategies:

- Migration-based versioing:
  1. Store SQL migration files (ex. v1_init.sql, v2_add_cols.sql)
  2. Tools: Flyaway, Liquidbase
  3. incremental, auditable

- Semantic Versioing:
  1. MAJOR → Breaking schema change
  2. MINOR → Additive change (new column)
  3. PATCH → Non-breaking fix

- Branching Strategy:
  1. Feature branches for schema Updates
  2. Pull request review before migration merge
  3. Tag release with schema version

- Backward Compatibility Policy.
  1. Add columns first
  2. Deprecate old fields
  3. Remove only after stable release

The schema version must be tied to the application version in Continuos Integration / Continuos Deployment or Continuos Delivery.

3. Design Jenkins pipelines for data validations workflows.

Ans Typical Jenkins pipelines stages:

1. Checkout Code
2. Build and containerize
3. Deploy to Test Environment
4. Run Data validation
   - Schema Validation
   - Run checks
   - Range checks
   - Duplication detection
   - statistical drift detection
5. Quality gate
   - Fail build if threshold exceeded
6. Promote to Production.

Ex.

```
stage('validation Data'){
    sh 'python validation.py'
}
```

4. Assess Selenium automation for data visualization UI.

Ans. Use Selenium to test:

- Dashboard loading performance
- Filter behaviour
- Chart rendering correctness
- Data refresh events
- Export functionality.

Validation approachs:

- DOM element precense
- Screenshot comparistm
- Data extraction from chart tooltips
- APP + UI cross validation

Limitations

- Fragile with dynamic stoit
- Slow for large database
- Requires stable locators

5. How Kubernetes ConfigMaps and secrets support Dyamic environments.

Ans. Kubernetes ConfigMaps and Secrets decouple configuration artifacts from image content, enabling the "build once, run anywhere" methodology. This seperation allows the same container image to function across development, staging, and production enviroments without modification.

① Mechanism of Injection:

ⓐ Environment Variables: Best for static configurations or flags. Data is injected during container setup.

ⓑ volumes: Best for large configuration files or certificates. Kubernetes mounts the config map/secret as a directory where each key becoms a file.

② Dynamic Capabilities:

ⓐ kubenet Polling: when a ConfigMap or Secret is update in the API server, the kubelet eventually detects the change and updates the file on the pod's filesystem

ⓑ Atomic Updates: Updates are handeled automatically through symlink swaps. Applications designed to watch for file changes can reload their configurations without a pod restart.

ⓒ Environment Variables Limitation: Unlike volumes, enviroment variables are immutable for the life of the process. updating the source ConfigMap will not reflect in the running contains environment until the pod is creating.

③ comparision of Roles

	ConfigMap	Secret
Data type	Non-sensitive	Sensitive
Storage	Plaintext in etcd	Base64 encoded
Size Limit	1MB	1MB

④ Integrate with CI/CD pipelines
In deployment workflows, ConfigMaps and Secrets allow for:
ⓐ Late Binding : Values are injected by the orchestrator at runtime, not the developer at build time
ⓑ Versioned Configurations : By using tools like Helm or Kustomize, team for version ConfigMaps. Changing the name name triggers a Rolling Update of Deployment ensuring a clean transition to new parameters.
ⓒ External Management : Secrets can be synchronized from external vaults into kubernetes secrets objects, ensuring security remains centralized while the deployment remains automated.

6. Node groups for enviroment-specific configuration.

Ans. Node groups (or node pools) allow you to portition a Kubernetes cluster into logical subsets based on hardware, location, or environment requirements. This enables precise control over where specified workloads are excecuted.

① The Mechanisms of Targeted Deployment.
To ensure environment-specific configuration land on the correct node groups, kubernetes uses three primary scheduling constraints.
ⓐ Node labels: Key-valued pairs attached to nodes
ⓑ Node selector: The simplest form of constraints, where a Pod's spec defined a required label that must be present on the node for scheduling to occur.
ⓒ Node Affinity: A more expressive language than node selectors allowing for "soft" or "hard" rules, as well as complex logic like In, NotIn or Exists.

② Strategic Use Case:
Node group support dynamic env by physializing logical boundories:

ⓐ Isolation of Workloads: Productions workloads can be restricted to high-performance, dedicated node groups, while development workloads use preemtiple (spot) increas to reduce costs.

ⓑ Compliance and Data Severeignty: Specific environments can be restricted to nodes with hardware-level encryption or specific geographics regions.

ⓒ Specialized Hardware: Enviroments requiring GPU acceleration or high-memory capacity can be isolated into specific pools, ensuring that general purpose workloads do not "starve" the expersive resources

③ Taints and Tolerants:

ⓐ Taints: Applied to a node group to ensure no pods are scheduled onto them unless the pod has matching Toleration.

ⓑ Enviroment Safeguarding: A common pattern is to taint "Production" nodes. Even if a developer, their "Dev" pod will not be scheduled on production nodes because it lacks the necessary toleration.

④ Implementation in Cloud Environments
In managed services (EKS, GKE, AKS), node group provide the between cloud infrastructure and kubernetes logic.

Feature

Auto-Scaling: Each node group can scale indepently based on enviroment-specific demand

IAM Integrating: Different node groups can be assigned different IAM

7. How Nagios notifications are generated and escalated

Ans Workflow:

① Service / Host Check
- Periodic monitoring via plugins

② Stage Check Detection
- OK → WARNING → CRITICAL

③ Event Handler Trigger

④ Notification Dispatch
- Email, SMS, Slack, PagerDuty

Estimation logic:
- Time-Based
- Severity-based
- Role-Based (L1 → L2 → L3 teams)

Configured in:
- Contacts
- contactgroups
- escalations
- notification_interval

8. Cloud-native Security in CI/CD pipelines.

Ans. Security is integrated via DevSecOps.

Integration points:
- Code Scanning
  SAST tools in build stage
- Dependency Scanning
  Detect vunerable libraries
- Container Image Scanning
  Trivy, Clair before push
- Secrets Detection
  Prevent API arc in Git

- Infrastructure as code Scanning
  Terraform / Cloud Formation checks
- Runtime Security
  Cloud-Native services

Code → Build → Security Scan → Test → Deploy

If the security check fails → block deployment.

Security becomes automated and continuous, not post deployment.